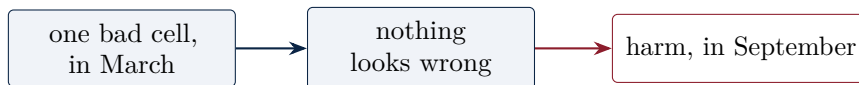


Lecture 07

Everyone Wants the Model Work

Data cascades, goodness of data, and the gate at the door



opaque, delayed, compounding, and usually avoidable

Recap of Lecture 06 in three points. Git keeps the five line receipt and DVC keeps the data. The fingerprint pins the exact file to the exact commit. Time travel is a pair: `git checkout` then `dvc checkout`.

We can now version data perfectly. A perfectly versioned dataset can still be perfectly bad.

Every pandas output in this handout is a real run on the course dataset. Both studies cited are named precisely, with figures taken from the papers rather than from summaries.

All sources verified 9 September 2026.

Contents

How this handout maps to the approved session plan	4
I Data Cascades	5
1 The clinic that turned patients away	5
2 The study that named the problem	5
3 How a cascade unfolds	6
4 The four common triggers, on our own project	6
5 Common errors: how teams invite cascades	6
II Goodness of Fit, Goodness of Data	7
6 The model that aced a broken exam	7
7 Two different kinds of good	7
8 What good data means for our project	7
9 Common errors in judging goodness	8
III The Quality Gate	9
10 The June file	9
11 The schema is the contract	9
12 What one text cell does to a column	9
13 The gate, in ten lines	10
14 What happens when the gate fails	10
15 Common errors at the gate	11
IV The Gate, in Its Production Form	12
16 Where the gate actually runs	12

17 The library form	12
V Consolidation	14
18 Module I is now complete	14
19 Assignment questions, as released	14
20 Check your understanding	15
21 Vocabulary introduced today	15
VI The Week Ahead	17
22 Seven days, about seven hours	17
VII Resource Library	19
23 For this lecture specifically	19
24 Carried forward	19

List of Figures

1	The shape of every cascade, redrawn from the properties described in the paper. The distance between the trigger and the harm is the reason nobody connects the two. . .	6
2	The four triggers, each mapped onto the delivery time project. None of them is exotic, and all four are present in a student project by default unless somebody decides otherwise.	6
3	Three ways to earn a great number that means nothing. A score is a statement about the relationship between a model and a dataset, not a statement about the world. . .	7
4	Type contagion from a single cell. Checked at the door, the error names the exact position. Unchecked, it becomes Tuesday's two in the morning mystery.	10
5	Columns, then types, then ranges. Each stage assumes the previous one passed, which is what makes the failure message specific enough to act on.	10
6	The three honest responses, and the one that feels like a fix. Quarantining keeps the evidence, which is what lets somebody find out why the vendor's file changed.	11
7	One contract, four checkpoints. This is why the schema is worth writing carefully now: almost every later lecture reuses it rather than inventing a new one.	12
8	What today added. The project can now refuse a file, which is a capability it did not have this morning and the only one that stops a cascade at its trigger.	14

How this handout maps to the approved session plan

Topic in this handout	Session plan location	Note
Data cascades in high stakes AI: physical world brittleness, domain drift	Session 13	The core of the session, with both source studies cited precisely
Goodness of fit against goodness of data	Session 13	As in the deck, extended with what each half is actually measured by
Data collection, ingestion and schema validation	Session 13	The quality gate. Extended with where the gate runs and what happens when it fails
Library support for validation	Beyond the plan	Added in Part IV, because the hand written gate is a teaching device and students should know its production form

The taught hour, minute by minute

0 to 4	Recap, then Section 1. Tell the clinic story before defining anything
4 to 14	Sections 2 and 3, the study that named the problem, and how a cascade unfolds
14 to 20	Section 4, the four triggers, mapped onto our own project
20 to 30	Sections 6 to 8, the broken exam, and the two kinds of good
30 to 42	Sections 10 to 13, the June file, the schema, and what one text cell does to a column
42 to 52	Sections 14 and 15, the gate itself and what happens when it fails
52 to 60	Section 18, Module I complete, and the closing line
Part IV, the library form of the gate, is the second slot or self study.	

PART I

Data Cascades

1 The clinic that turned patients away

Thailand's health ministry set a goal of screening sixty percent of its diabetic population for diabetic retinopathy, a condition that causes blindness if it is not caught early. The country has roughly 4.5 million diabetic patients and about 200 retinal specialists, so nurses do the screening: they photograph the retina and send the image for review.

A deep learning system was deployed across eleven clinics to give that assessment immediately. In the laboratory it was excellent. In the clinics, the rooms were dim and nurses photographed eyes in a hurry, and the system had been designed to *reject* any image whose quality fell below a threshold.

What actually happened

Patients were sent home unscreened, some after travelling a long way, because the system refused images it considered too poor rather than guessing on them.

Notice the uncomfortable part. Refusing to guess on a bad image is a *good* safety design. What made it a harm was the gap between the conditions it was built for and the conditions it met.

The model was fine. The data assumption broke, months earlier, in a different building.

2 The study that named the problem

Researchers interviewed 53 practitioners building high stakes systems, in India, East and West African countries, and the United States, working in domains including cancer detection, landslide detection and suicide prevention.

Data cascade, as defined in the paper

Compounding events causing negative, downstream effects from data issues, triggered by conventional practices that undervalue data quality.

Two numbers from that study are worth memorising. **92 percent** of the practitioners had experienced one or more cascades. And **45.3 percent** reported *two or more*, which tells you these are not rare accidents but a standing condition of the work.

3 How a cascade unfolds

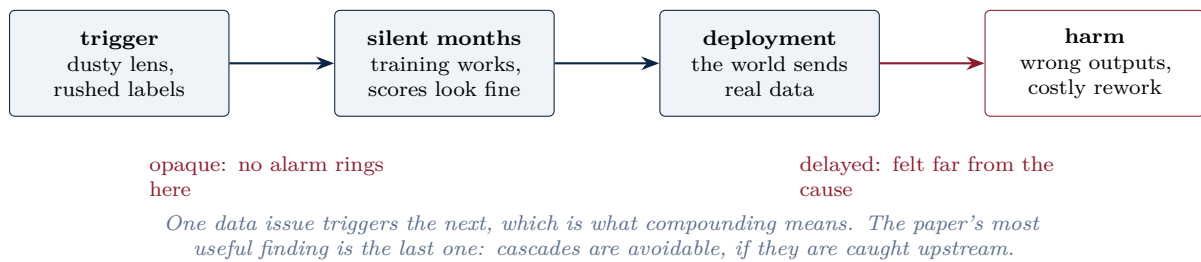
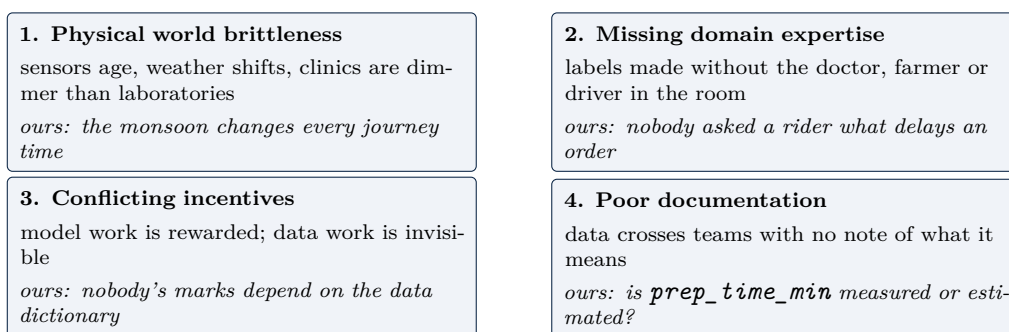


Figure 1: The shape of every cascade, redrawn from the properties described in the paper. The distance between the trigger and the harm is the reason nobody connects the two.

4 The four common triggers, on our own project



The paper's title says the rest: everyone wants to do the model work, not the data work. Trigger three is the one that produces the other three.

Figure 2: The four triggers, each mapped onto the delivery time project. None of them is exotic, and all four are present in a student project by default unless somebody decides otherwise.

5 Common errors: how teams invite cascades

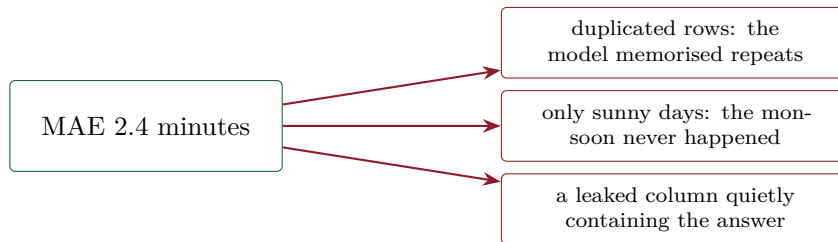
The error	What actually happens
Treating data as somebody else's job	Nobody owns the dataset, so nobody notices it rotting
Believing a good test score means good data	The score only ever sees the data you gave it
Silently deleting awkward rows	The problem is hidden rather than fixed, and now undocumented
Documenting nothing at handover	The next team inherits the data and none of the warnings

PART II

Goodness of Fit, Goodness of Data

6 The model that aced a broken exam

Our delivery model scores a mean absolute error of 2.4 minutes. Celebration?



Every one of these produces an excellent score. None of them is visible in the score, because the score is computed on the same broken data.

Figure 3: Three ways to earn a great number that means nothing. A score is a statement about the relationship between a model and a dataset, not a statement about the world.

7 Two different kinds of good

	Goodness of fit	Goodness of data
The question	How well does the model match the data it was given	How far does the data deserve to be trusted at all
Measured by	MAE, accuracy, precision	complete, fresh, representative, correctly labelled
Effort	one line of code	checked deliberately, column by column
Who notices if it is missing	everyone, immediately	nobody, for months

The paper's warning

Teams measured goodness of fit and stopped there. Cascades grow in the unmeasured half.

8 What good data means for our project

For our file of orders, good means four checkable things.

Property	What it means here
Complete	Five columns, no missing cells
Sane ranges	Distance positive, traffic between 1 and 3, rain either 0 or 1
Representative	Rainy days and far suburbs included, not just easy orders
Fresh	This season's city, not last year's

Each of these is checkable, and none of them appears anywhere in the MAE.

9 Common errors in judging goodness

The error	The correction
Polishing the model on broken data	Tuning cannot fix rows that are wrong
Comparing scores across different data versions	The receipts from Lecture 06 exist precisely so you never do this blind
Treating more data as the fix for bad data	More of the same flaw is a bigger flaw
Testing on data the world no longer sends	A fresh score on stale rows is a stale score

PART III

The Quality Gate

10 The June file

Every month the vendor emails a fresh orders file. In June, one operator typed a delivery time as text:

```
...,3.2,12,2,0,45 mins
```

One cell, one human, one hurry. Nothing crashes. Training reads the file at two in the morning next Tuesday.

The house rule

Files from outside are guests, not family. They are checked at the door, every visit.

11 The schema is the contract

Schema

The agreed shape of a data file: which **columns**, in what **types**, within what **ranges**. If the file breaks the contract, it does not enter.

For our project the contract says: exactly five columns, in order; four numeric predictors and one numeric target; `traffic_level` between 1 and 3, `rain` either 0 or 1, and `distance_km` greater than zero.

12 What one text cell does to a column

```
import pandas as pd
df = pd.read_csv("delivery_times.csv")
print(df.dtypes)
# distance_km      float64
# prep_time_min    int64
# traffic_level     int64
# rain             int64
# delivery_min     float64

dfj = pd.read_csv("delivery_june.csv")
print(dfj.dtypes)
# delivery_min      object      <-- was float64

pd.to_numeric(dfj["delivery_min"])
# ValueError: Unable to parse string "45 mins" at position 17
```



This is why the type check is not pedantry. A single bad cell does not corrupt one row; it changes what pandas believes the entire column is, and every later calculation inherits that belief.

Figure 4: Type contagion from a single cell. Checked at the door, the error names the exact position. Unchecked, it becomes Tuesday's two in the morning mystery.

13 The gate, in ten lines

```

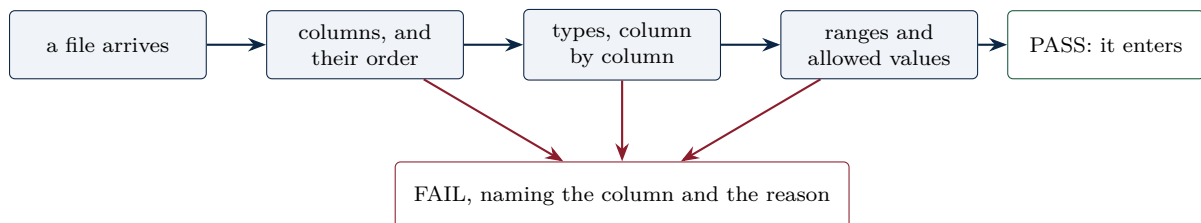
SCHEMA = {"distance_km": "float64", "prep_time_min": "int64",
          "traffic_level": "int64", "rain": "int64",
          "delivery_min": "float64"}

def gate(path):
    df = pd.read_csv(path)
    assert list(df.columns) == list(SCHEMA), "wrong columns"
    for col, want in SCHEMA.items():
        assert str(df[col].dtype) == want, f"{col} is {df[col].dtype}"
    assert df["traffic_level"].between(1, 3).all()
    assert df["rain"].isin([0, 1]).all()
    assert (df["distance_km"] > 0).all()
    print(f"PASS {path} ({len(df)} rows)")
  
```

```

gate("delivery_times.csv")
# PASS delivery_times.csv (600 rows)

gate("delivery_june.csv")
# AssertionError: delivery_min is object
  
```

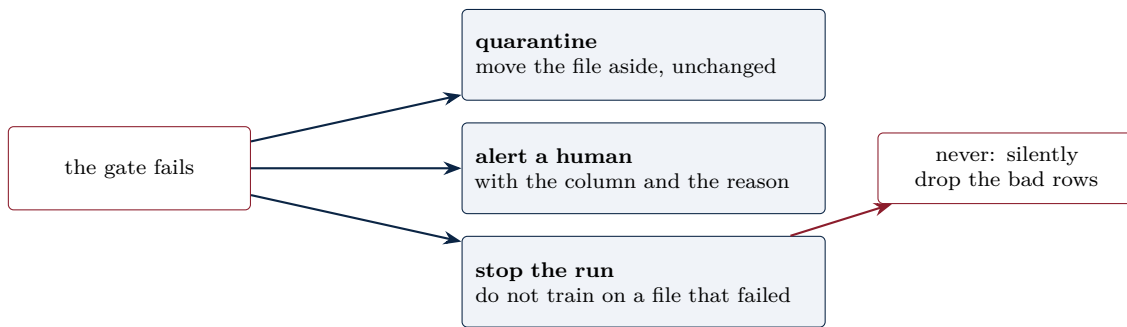


The order matters. Checking ranges before types would compare numbers against strings, and the message you get back would be about the wrong thing entirely.

Figure 5: Columns, then types, then ranges. Each stage assumes the previous one passed, which is what makes the failure message specific enough to act on.

14 What happens when the gate fails

The deck stops at the failure message. In a real pipeline the next decision is the important one, and there are only three honest options.



Dropping the offending rows makes the error message go away and leaves the cascade running, now with no record that anything was ever wrong. That is a new trigger, not a fix.

Figure 6: The three honest responses, and the one that feels like a fix. Quarantining keeps the evidence, which is what lets somebody find out why the vendor's file changed.

Why `errors="coerce"` is usually the wrong choice in a gate

`pd.to_numeric(column, errors="raise")` stops and names the offending value. `errors="coerce"` turns every unparseable cell into a missing value and carries on.

The second one makes your gate pass. It does not make your data good: it converts a loud, locatable error into a silent one, and the cascade continues with the evidence destroyed. Coercion belongs in exploratory work, where you are looking at the result. It does not belong at the door.

15 Common errors at the gate

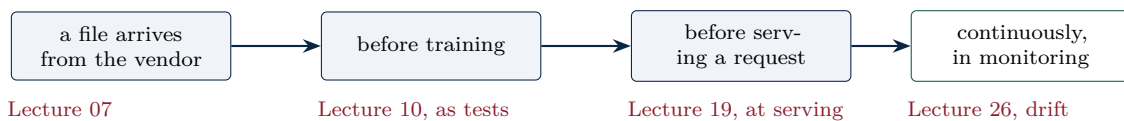
The error	What actually happens
Reading the file and never looking at the types	The object column trains as garbage, or crashes much later
Using coercion to make the problem vanish	Bad cells silently become missing values, and the cascade continues invisibly
Dropping bad rows without logging	Fixing by deletion is a cascade trigger of its own
Validating once, ever	The gate runs on every arrival, or it is decoration

PART IV

The Gate, in Its Production Form

This part is beyond the released slides. The ten line gate is a teaching device, and it is worth knowing what it grows into.

16 Where the gate actually runs



The same contract, enforced at four different moments. A schema written once in this lecture will reappear as a pytest test, as a request model on the API, and as the reference window for drift detection.

Figure 7: One contract, four checkpoints. This is why the schema is worth writing carefully now: almost every later lecture reuses it rather than inventing a new one.

17 The library form

Hand written asserts are perfect for learning and awkward at scale: they stop at the first failure, and the contract is scattered through the code rather than written down in one place. The library answer is a declared schema.

```

import pandera.pandas as pa

schema = pa.DataFrameSchema(
    {
        "distance_km": pa.Column(float, pa.Check.gt(0)),
        "prep_time_min": pa.Column(int, pa.Check.between(1, 120)),
        "traffic_level": pa.Column(int, pa.Check.isin([1, 2, 3])),
        "rain": pa.Column(int, pa.Check.isin([0, 1])),
        "delivery_min": pa.Column(float, pa.Check.gt(0)),
    },
    ordered=True,
)

schema.validate(df, lazy=True)

```

What it adds	Why it matters
<code>ordered=True</code>	Checks column order as well as presence, which matters whenever a frame is handed to a library that reads positions
<code>lazy=True</code>	Collects <i>all</i> failures rather than stopping at the first, so one run tells you everything wrong with the file
Failure cases as a table	Reports which row, which column, which rule and the offending value
A schema object	The contract is one declared thing you can import, version and reuse at all four checkpoints

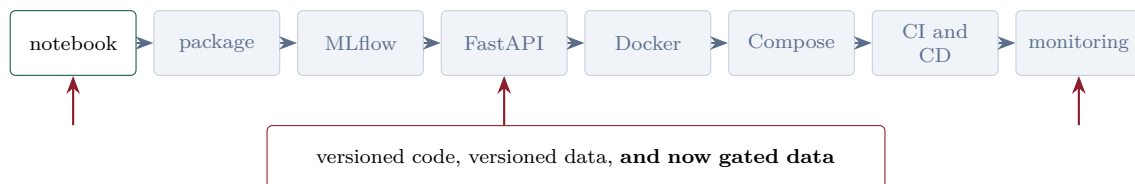
Which to learn

Write the ten line gate first, by hand, so you understand what is being checked and in what order. Then move it into a declared schema once the contract stops fitting comfortably in your head. Great Expectations is the heavier alternative, worth knowing if your data lives in a warehouse rather than a file.

PART V

Consolidation

18 Module I is now complete



Every arriving file is now checked against the schema before a single row is trained on. Versioned data from Lecture 06 plus gated data from Lecture 07 is the data foundation of Module I, complete.

Figure 8: What today added. The project can now refuse a file, which is a capability it did not have this morning and the only one that stops a cascade at its trigger.

19 Assignment questions, as released

Set 1 • Data cascades • CO1

Q1 (Understand). Define a data cascade in your own words, and explain the two properties that make cascades hard to catch: they are *opaque* and *delayed*.

Q2 (Apply). Match each failure to its trigger, choosing from physical world brittleness, missing domain expertise, conflicting incentives, and poor documentation. (a) A crop disease application fails after harvest season changes the leaves. (b) Chest radiographs labelled by interns without a radiologist. (c) A column named `temp` means temporary to one team and temperature to another.

Q3 (Build). For our delivery time project, write a one page cascade risk register: three ways the data could silently go wrong, for example the vendor changing units, the service area expanding, or the monsoon shifting traffic, and for each, the earliest signal we could watch for.

Set 2 • Goodness of data • CO1

Q1 (Understand). Differentiate goodness of fit from goodness of data, with one example measure of each. Why can a model have excellent fit on data of terrible goodness?

Q2 (Apply). After the vendor's new file arrives, MAE improves from 3.1 to 2.2 minutes. In three sentences, list the checks you would run *before* celebrating. Think about what changed between the two data versions, and whether any rows are repeated or easier.

Q3 (Build). Write a five item data goodness checklist for our file, with one line of reasoning each. Then run two of the checks in pandas, the total row count and the duplicated row count, and report both numbers.

Set 3 • Schema validation • CO1

Q1 (Understand). Explain what a schema is for a data file, and why `pd.to_numeric(..., errors="raise")` belongs in a quality gate while `errors="coerce"` usually does not.

Q2 (Apply). A vendor file has one `delivery_min` cell containing the text 45 mins. Predict what

`df.dtypes` shows for that column after reading the file, and state which line of today's `gate()` stops it.

Q3 (Build). Extend today's `gate()` with two more checks: `prep_time_min` between 1 and 120, and zero duplicated rows. Run it on the good file, which should pass, then break a copy yourself and paste the failing message.

20 Check your understanding

1. In the clinic story, what exactly failed, and why is calling it a model failure wrong?
2. Why is refusing a poor quality image both a good design and, in context, a harm?
3. Give the definition of a data cascade, and the two numbers from the study.
4. Name the two properties that make cascades hard to catch, and say which stage of the timeline each describes.
5. Name the four triggers, and give one example of each from our own project.
6. Which of the four triggers tends to produce the other three?
7. Give three ways a model can earn an excellent score on bad data.
8. State the difference between goodness of fit and goodness of data in one sentence each.
9. One cell in a column contains text. What happens to the column, and why is that worse than one bad row?
10. Give the three stages of the gate in order, and say why the order matters.
11. Your gate fails on the vendor's file. Name the three honest responses and the one that is not.
12. Why is coercion the wrong choice at the door but acceptable during exploration?
13. Name two things a declared schema gives you that a list of asserts does not.

21 Vocabulary introduced today

Term	Meaning
Data cascade	Compounding events caused by data issues, whose damage surfaces downstream, long after the cause
Opaque	The cascade gives no clear indicator at the moment it is created
Delayed	The harm appears far from the trigger, in time and in place
Physical world brittleness	The assumption that deployment conditions match training conditions, broken by reality
Goodness of fit	How well a model matches the data it was given
Goodness of data	How far the data deserves to be trusted at all
Schema	The agreed shape of a data file: columns, types, ranges
Quality gate	The check that a file must pass before it is allowed into the project
dtype	The type pandas has detected for a column
Type contagion	One bad cell changing the detected type of an entire column

Term	Meaning
Quarantine	Moving a failing file aside, unchanged, rather than fixing or deleting it
Declared schema	The contract written once as an object, importable and reusable at every checkpoint

PART VI

The Week Ahead

22 Seven days, about seven hours

Day 1 • Read the paper that named it • 75 minutes

Read. Sambasivan et al., *Everyone wants to do the model work, not the data work: Data Cascades in High Stakes AI*, CHI 2021, open access. Read the properties of cascades and the four trigger sections. <https://doi.org/10.1145/3411764.3445518>

Produce. One paragraph on which of the four triggers you think is most present in student projects, with a reason.

Day 2 • Read the clinic study • 60 minutes

Read. Beede et al., *A Human Centered Evaluation of a Deep Learning System Deployed in Clinics for the Detection of Diabetic Retinopathy*, CHI 2020. <https://doi.org/10.1145/3313831.3376718>

Produce. Two sentences on the tension the authors describe between the model's thresholds for data quality and the quality of data that a resource constrained clinic can actually produce.

Day 3 • Learn what pandas thinks your columns are • 45 minutes

Read. The dtypes section of the pandas user guide. https://pandas.pydata.org/docs/user_guide/basics.html

Produce. Print the dtypes of our file, then deliberately put a word into one numeric cell, reread it, and print them again. Record both.

Day 4 • Build the gate • 60 minutes

Do. Assignment Set 3, Q3: extend the gate with the range check and the duplicate check, pass the good file, then break a copy.

Produce. Both outputs, the pass and the failure message. A gate you have never seen fail is a gate you do not know works.

Day 5 • Write the risk register • 60 minutes

Do. Assignment Set 1, Q3: three ways our data could silently go wrong, and the earliest signal for each.

Why this matters. The earliest signal column is the specification for the monitoring you will build in Lecture 26. Write it now and that lecture becomes implementation rather than invention.

Day 6 • The library form • 45 minutes

Read and do. The pandera documentation on dataframe schemas and data type validation.
https://pandera.readthedocs.io/en/stable/dataframe_schemas.html

Produce. Rewrite your gate as a declared schema, run it with `lazy=True` on your broken copy, and compare the report with the single assert failure you got on Day 4.

Day 7 • Document the data • 45 minutes

Do. Write the data dictionary for our five columns: meaning, units, source, and one line on how each could go wrong. Commit it in the repository beside the schema.

Why this matters. Trigger four is poor documentation. This file is the direct countermeasure, and it takes under an hour once in the life of a project.

If you only have one hour all week

Day 4, then Day 5. Building a gate and watching it fail, then writing down the three ways your own data could betray you. Those two are the entire lecture, in practice.

PART VII

Resource Library

All links verified on 9 September 2026.

23 For this lecture specifically

Resource	Where, and why
Sambasivan et al., Data Cascades in High Stakes AI, CHI 2021, open access	https://doi.org/10.1145/3411764.3445518 53 practitioners, 92 percent prevalence, 45.3 percent with two or more. The paper this lecture is built on
Beede et al., A Human Centered Evaluation of a Deep Learning System Deployed in Clinics, CHI 2020	https://doi.org/10.1145/3313831.3376718 Eleven clinics in Thailand, and the tension between model quality thresholds and real clinic conditions
pandas user guide, dtypes	https://pandas.pydata.org/docs/user_guide/basics.html The reference for every type check in today's gate
pandera documentation	https://pandera.readthedocs.io/en/stable/dataframe_schemas.html The declared schema form of the gate, with ordered and lazy validation

24 Carried forward

Resource	Where
Sculley et al., Hidden Technical Debt in Machine Learning Systems	https://proceedings.neurips.cc/paper_files/paper/2015/file/86df7dcfd896fc2674f757a2463eba-Paper.pdf Unstable data dependencies and correction cascades are the same family of problem
Kapoor and Narayanan, leakage and the reproducibility crisis	https://doi.org/10.1016/j.patter.2023.100804 What happens to a whole field when the data half goes unchecked
Amershi et al., Software Engineering for Machine Learning, ICSE 2019	https://www.microsoft.com/en-us/research/wp-content/uploads/2019/03/amershi-icse-2019_Software_Engineering_for_Machine_Learning.pdf
Get Started with DVC	https://dvc.org/doc/start

Resource	Where
scikit-learn, common pitfalls and recommended practices	https://scikit-learn.org/stable/common_pitfalls.html
Machine Learning in Production, DeepLearning.AI, free to audit	https://www.coursera.org/learn/introduction-to-machine-learning-in-production
MLOps DataTalks.Club	Zoomcamp, https://github.com/DataTalksClub/mlops-zoomcamp
Made With ML	https://madewithml.com
Stanford CS 329S	https://stanford-cs329s.github.io

Data work is the model work.

Next lecture. Lecture 08, Module I consolidation, on the Socio-Ed platform. Production mindset, versioned code, versioned data, gated data: the whole module replayed, with a recap video, a discussion forum and a practice quiz that carries no marks. Bring nothing, and come with questions.